Detección de ruido mediante DBSCAN: descripción del código y estrategias de paralelización con OpenMP

Proyecto Apertura · Cómputo Paralelo y en la Nube · Instituto Tecnológico Autónomo de México (ITAM), Otoño 2026
Profesor: Dr. J. Octavio Gutiérrez García · 20 de octubre de 2026
Integrantes: Mario Guillen Blanco · Gabriel Navarro Cerón

Resumen

Este documento describe el código del proyecto y las estrategias de paralelización: el detector de ruido de dos pasos basado en DBSCAN ($\varepsilon = 0.03$, `min_samples` $= 10$) implementado en C++ en tres versiones dentro de `dbscan.cpp`: una serial de búsqueda exhaustiva; P1, que reparte los puntos entre hilos de OpenMP ("toda la matriz indivisible"); y P2, que divide el plano en k regiones con halos de ancho ε ("dividiendo la matriz por k"). La evaluación experimental de desempeño se presenta en un documento aparte.

1. Problema y formulación matemática

El enunciado pide detectar ruido en una nube de puntos del plano con la variante "ingenua" de DBSCAN vista en clase [1, 3]. A diferencia del DBSCAN completo, no se forman ni se numeran grupos: cada punto recibe una de tres clases mediante dos pasos. Sea $P = \{p_1, \ldots, p_N\}$ con $p_i = (x_i, y_i) \in \mathbb{R}^2$, y sean $\varepsilon = 0.03$ y $m =$ `min_samples` $= 10$ (los parámetros de todas las ejecuciones oficiales). El vecindario

$$\mathcal{N}_\varepsilon(i) = \left\{ j \in \{1, \ldots, N\} : \ \|p_i - p_j\|_2 \le \varepsilon \right\} \tag{1}$$

incluye al propio i (a distancia 0) y a los duplicados, igual que scikit-learn [6]. Las clases son

$$\text{Paso 1 (core de primer grado):} \quad C_1 = \left\{ i : \ |\mathcal{N}_\varepsilon(i)| \ge m \right\}, \tag{2}$$

$$\text{Paso 2 (core de segundo grado):} \quad C_2 = \left\{ i \notin C_1 : \ \exists\, j \in C_1 \text{ con } j \in \mathcal{N}_\varepsilon(i) \right\}, \tag{3}$$

$$\text{Ruido:} \quad R = \{1, \ldots, N\} \setminus (C_1 \cup C_2), \tag{4}$$

codificadas como 1 (C_1, el "core" de la literatura), 2 (C_2, el punto "borde") y 0 (ruido). El Paso 2 es de un solo salto: (3) solo consulta C_1, que está completo antes de empezar. Un punto de C_2 nunca convierte a otro en C_2; no hay la propagación recursiva con la que el DBSCAN completo expande sus grupos.

Distancia al cuadrado. Como $\varepsilon > 0$ y $t \mapsto t^2$ es estrictamente creciente en $[0, \infty)$,

$$\|p_i - p_j\|_2 \le \varepsilon \iff (x_i - x_j)^2 + (y_i - y_j)^2 \le \varepsilon^2, \tag{5}$$

así que el programa compara distancias al cuadrado y nunca calcula raíces. En doble precisión, los pares a distancia casi exactamente ε (frecuentes con tres decimales) los decide el redondeo; por eso las tres versiones usan una única función (`within_eps`, Listado 1), compilada sin `-ffast-math` ni fusión de multiplicación y suma (`-ffp-contract=off`), y toman las mismas decisiones.

Entrada y salida. La entrada es el CSV sin encabezado de la libreta `DBSCAN_noise.ipynb` (`x,y` por línea). Con `--output` el programa escribe `x,y,etiqueta` en el orden de entrada (1 = no ruido, 0 = ruido, como `np.where(clusters==-1, 0, 1)`), que grafica la segunda celda de la libreta; con `--detailed-output`, `x,y,clase,conteo`; y en la salida estándar, una línea `METRICS` con tiempos y huellas digitales (FNV-1a de 64 bits) de las clases y los conteos. La etiqueta binaria no distingue los dos tipos de core: ambos valen 1. Con 4,000 puntos comprobamos que la segunda celda de la libreta original, sin cambios en su lógica, lee este archivo con `np.loadtxt` y lo grafica (documento *Evaluación experimental de desempeño*, sección 2.1).

2. Implementaciones: Serial, P1 y P2

Las tres versiones están en `dbscan.cpp` (`--mode serial|p1|p2`) y comparten el predicado y dos núcleos (Listado 1): `count_neighbors` cuenta, sin salida anticipada, los candidatos a distancia $\le \varepsilon$, y

`has_core_neighbor` indica si alguno de C_1 lo está y se detiene en el primero. Solo difieren en *qué candidatos* recibe cada consulta y *qué hilo* la ejecuta. Ninguna guarda una matriz de distancias (con $N = 200{,}000$ ocuparía 320 GB); la memoria es $O(N)$. Los resultados se escriben por índice original en `count`, `is_core` y `label`, con banderas `unsigned char`: un `std::vector<bool>` guarda ocho banderas por byte y haría que dos hilos escribieran el mismo byte.

Listado 1: Predicado y núcleo del Paso 1, compartidos por las tres versiones.

```
inline bool within_eps(double dx, double dy, double eps2) {   // eps2 = eps * eps
    return dx * dx + dy * dy <= eps2;
}
int count_neighbors(double xq, double yq, const double* xs, const double* ys, int n, double eps2) {
    int count = 0;           // el punto consultado es uno de los candidatos: se cuenta a sí mismo
    for (int j = 0; j < n; j++)
        if (within_eps(xs[j] - xq, ys[j] - yq, eps2)) count++;
    return count;
}
```

2.1. Serial: búsqueda exhaustiva

`dbscan_serial` hace dos recorridos secuenciales: el Paso 1 cuenta los vecinos de cada i entre los N puntos y marca `is_core[i]`; ya con todas las banderas, el Paso 2 asigna 1, 2 (si `has_core_neighbor` encuentra un punto de C_1) o 0. El Paso 1 evalúa exactamente N^2 veces el predicado: el costo es $O(N^2)$. El Paso 2 casi no cuesta con nuestros datos (3 ms de 8.68 s con $N = 200{,}000$) porque casi todos los puntos están en C_1. Serial no usa regiones ni directivas de OpenMP (solo `omp_get_wtime()` para medir), y el programa rechaza `--k` y cualquier `--threads` distinto de 1.

2.2. P1: paralelización por puntos (“toda la matriz indivisible”)

P1 conserva la búsqueda exhaustiva y reparte las iteraciones externas entre hilos (Listado 2): cada hilo clasifica puntos distintos y todos leen el arreglo completo. `schedule(static)` divide el ciclo desde el inicio en bloques contiguos de unas N/T iteraciones, sin costo de reparto; es razonable porque cada iteración recorre los mismos N candidatos. Se eligió con una medición previa: `dynamic,64` fue un 5 % más rápido con 12 hilos, pero sin superar el umbral fijado de antemano (dos desviaciones estándar), y la regla eligió la opción más simple.

El Paso 2 es otro `parallel for` que lee `is_core` y escribe `label[i]`; la barrera implícita al final del primer ciclo garantiza que C_1 está completo. No hay condiciones de carrera: cada posición i de `count`, `is_core` y `label` la escribe solo su iteración, sin `critical`, `atomic` ni candados. Antes de medir, `start_threads` llama a `omp_set_dynamic(0)` (impide usar menos hilos de los pedidos; no es `schedule(dynamic)`) y crea el equipo de hilos. P1 sigue evaluando los N^2 pares, solo que repartidos entre T hilos.

Listado 2: Paso 1 de P1. Cada iteración escribe únicamente las posiciones de su punto.

```
#pragma omp parallel for default(none) \
        shared(n, x, y, count, is_core, eps2, min_samples) schedule(static)
for (int i = 0; i < n; i++) {
    count[i]   = count_neighbors(x[i], y[i], x, y, n, eps2);   // todos los puntos son candidatos
    is_core[i] = count[i] >= min_samples;
}   // barrera implícita: ningún hilo empieza el Paso 2 antes de que is_core esté completo
```

2.3. P2: regiones del plano con halos (“dividir la matriz por k”)

P2 divide la caja envolvente en $k = r \times c$ rectángulos iguales (r: el mayor divisor de k que no supera $\sqrt{k}$; $k = 2$: mitades verticales, $k = 4$: cuadrantes, $k = 64$: 8×8), como en el dibujo del enunciado. La **región propietaria** de i es la celda que contiene a p_i; la asigna `strip_index`, que nunca disminuye cuando la coordenada crece, así que cada punto tiene una sola dueña. El **halo** de la región r son los puntos de otras regiones a distancia $\leq \varepsilon$ de su rectángulo en cada eje, incluidas las diagonales (Figura 1): cada punto j entra al halo de todas las regiones que toca el cuadrado $[x_j - \mu, x_j + \mu] \times [y_j - \mu, y_j + \mu]$, con $\mu = \varepsilon + 10^{-9}(M + \varepsilon)$ y M la mayor coordenada en valor absoluto (el margen, $\approx 10^{-9}$, solo absorbe

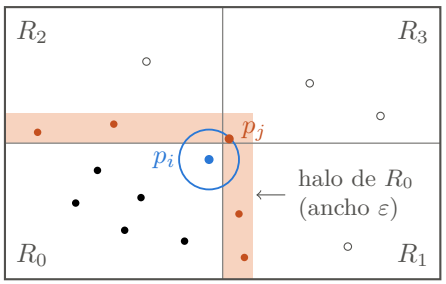


Figura 1: P2 con $k = 4$. La región R_0 compara sus puntos propios (negro) con ellos mismos y con los de su halo (naranja): la franja sombreada de ancho ε en las regiones vecinas, incluida la esquina diagonal. El vecino p_j de p_i pertenece a R_3, pero está en el halo de R_0. Los puntos huecos nunca se comparan con los de R_0: ese es el ahorro de trabajo de P2.

Listado 3: Paso 1 de P2: una tarea por región, de la más costosa a la menos costosa.

```
#pragma omp parallel for default(none) \
        shared(k, ord, reg, count, is_core, min_samples, eps2) schedule(dynamic, 1)
for (int o = 0; o < k; o++) {
    const Region& R = reg[ord[o]];          // ord: regiones en orden descendente de costo
    const int L = (int)R.lid.size();        // candidatos: propios + halo
    for (int a = 0; a < R.n_owned; a++) {   // solo se clasifican los puntos propios
        const int i = R.lid[a];             // índice original del punto
        count[i]   = count_neighbors(R.lx[a], R.ly[a], R.lx.data(), R.ly.data(), L, eps2);
        is_core[i] = count[i] >= min_samples;
    }
}
```

redondeo). La **lista local de candidatos** L_r tiene los n_r puntos propios seguidos del halo, copiados a arreglos contiguos con su índice original.

Por qué el halo no pierde vecinos. Como $|x_i - x_j| \le \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$, y lo mismo en y,

$$\|p_i - p_j\|_2 \le \varepsilon \implies |x_i - x_j| \le \varepsilon \quad \text{y} \quad |y_i - y_j| \le \varepsilon. \tag{6}$$

Entonces $x_j - \mu \le x_i \le x_j + \mu$; como `strip_index` es monótona y es la *misma* función que define la región propietaria, la columna de la región de i está en el rango que alcanza j, e igual con las filas: todo vecino real de un punto propio está en L_r. Cada punto aparece una sola vez en L_r y los candidatos de más se descartan con el predicado exacto (5), así que los conteos son idénticos a los de Serial (el margen μ cubre el redondeo en el dominio numérico que el programa acepta).

Fases. (i) *Construcción* (`t_build`): caja, región propietaria y alcance de cada punto, y listas L_r (trabajo $O(kN)$). (ii) *Paso 1* (Listado 3): cada región cuenta los vecinos de sus puntos propios contra L_r y escribe `count` e `is_core` en el índice original. (iii) *Paso 2*: cada región copia `is_core` de sus candidatos —así recibe las banderas de su halo, calculadas por otras regiones— y clasifica sus puntos propios. Al escribir en el índice original, la unión es inmediata y se conserva el orden de entrada.

El Paso 2 no necesita un halo de 2ε. Basta un $j \in C_1$ con $\|p_i - p_j\|_2 \le \varepsilon$, el radio del Paso 1, así que j ya está en L_r por (6). Su bandera no se recalcula (eso sí exigiría conocer puntos a 2ε de i): la calculó la región dueña de j en el Paso 1 y está completa en `is_core` después de la barrera.

Concurrencia y planificación. Solo la región dueña escribe `count`, `is_core` y `label` de un punto, una vez; las listas de una región solo las modifica su iteración (el arreglo de regiones se dimensiona antes); en el Paso 2 `is_core` solo se lee; y las barreras implícitas separan las fases. Cada región es una tarea de `schedule(dynamic,1)`. Como los datos forman grupos, las cargas son desiguales (con $N = 200{,}000$ y $k = 64$ solo 55 regiones tienen puntos y la más costosa tiene el 14.7 % de los pares), y por eso las tareas se ordenan de mayor a menor costo $n_r|L_r|$ (“tareas más largas primero” [5]).

Trabajo. P2 evalúa $\sum_r n_r|L_r| \le N^2$ pares (`pairs_step1`). El ahorro depende de los datos, de la cuadrícula y de ε; no es una complejidad subcuadrática garantizada (ver la sección de limitaciones del documento *Evaluación experimental de desempeño*).

Alternativa evaluada: P2 por bloques. La primera P2 del equipo dividía por índices la matriz de comparaciones en k bloques con conteos parciales unidos por suma y por “o” lógico. Reprodujo a Serial

en todas las pruebas, pero evalúa los mismos N^2 pares con a lo más k tareas; en una comparación previa (200,000 puntos, 12 hilos) la versión espacial fue 5.2 ($k = 32$) y 6.0 ($k = 64$) veces más rápida. Esa versión no forma parte de esta entrega; la versión oficial de P2 es la espacial descrita arriba.

Referencias

[1] J. O. Gutiérrez García. *Proyecto Apertura 2026B: DBSCAN* (enunciado) y libreta `DBSCAN_noise.ipynb`. ITAM, 2026.

[2] J. O. Gutiérrez García. *Introducción y arquitecturas paralelas* (diapositivas del curso), ITAM, 2026.

[3] M. Ester, H.-P. Kriegel, J. Sander y X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. En *Proc. KDD-96*, pp. 226–231, 1996.

[4] G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. En *AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967.

[5] R. L. Graham. Bounds on multiprocessing timing anomalies. *SIAM J. Appl. Math.*, 17(2):416–429, 1969.

[6] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *JMLR*, 12:2825–2830, 2011.

[7] IEEE Robotics and Automation Society. *Guidelines for Generative AI Usage*. `https://www.ieee-ras.org/publications/guidelines-for-generative-ai-usage`.

[8] Elsevier. *Generative AI policies for journals*. `https://www.elsevier.com/about/policies-and-standards/generative-ai-policies-for-journals`.